Experiment 01

Clean, Integrate and Transform EHR Data

In [1]:
```
import pandas as pd
import numpy as np
```

1. Load Healthcare Datasets

In [2]:
```
patients = pd.read_csv("patients.csv")
conditions = pd.read_csv("conditions.csv")
```

In [3]:
```
print("Patients Dataset Shape:", patients.shape)
print("Conditions Dataset Shape:", conditions.shape)
```

```
Patients Dataset Shape: (1163, 25)
Conditions Dataset Shape: (38094, 6)
```

2. Display Original Data

In [5]:
```
print("\nPatients Dataset:")
display(patients.head())
```

```
Patients Dataset:
```

	Id	BIRTHDATE	DEATHDATE	SSN	D
0	b9c610cd-28a6-4636-ccb6-c7a0d2a4cb85	2019-02-17	NaN	999-65-3251	
1	c1f1fcaa-82fd-d5b7-3544-c8f9708b06a8	2005-07-04	NaN	999-49-3323	S9
2	339144f8-50e1-633e-a013-f361391c4cff	1998-05-11	NaN	999-10-8743	S9
3	d488232e-bf14-4bed-08c0-a82f34b6a197	2003-01-28	NaN	999-56-6057	S9
4	217f95a3-4e10-bd5d-fb67-0cfb5e8ba075	1993-12-23	NaN	999-91-4320	S9

5 rows × 25 columns

In [6]:
```
print("\nConditions Dataset:")
display(conditions.head())
```

```
Conditions Dataset:
```

	START	STOP	PATIENT	ENCOU
0	2013-06-24	2013-07-02	c1f1fcaa-82fd-d5b7-3544-c8f9708b06a8	0b2794bd-ec2b-d34f-0610-2523b3
1	2016-02-27	2016-03-14	c1f1fcaa-82fd-d5b7-3544-c8f9708b06a8	a6d818dd-0983-fd1c-eefa-3d22955
2	2016-08-11	2016-08-22	c1f1fcaa-82fd-d5b7-3544-c8f9708b06a8	36d2e781-4655-0a11-1f70-c69856e
3	2016-11-27	2016-12-17	c1f1fcaa-82fd-d5b7-3544-c8f9708b06a8	c8eaaf41-958b-31ab-7de5-568cee8
4	2017-02-22	2017-06-02	c1f1fcaa-82fd-d5b7-3544-c8f9708b06a8	6474f606-5a1b-48c0-bbbf-ad6dcbc

3. Check Missing Values

In [7]:
```
print("\nMissing Values in Patients Dataset:")
print(patients.isnull().sum())
```

```
Missing Values in Patients Dataset:
Id                          0
BIRTHDATE                   0
DEATHDATE                1000
SSN                         0
DRIVERS                   215
PASSPORT                  276
PREFIX                    245
FIRST                       0
LAST                        0
SUFFIX                   1147
MAIDEN                    832
MARITAL                   384
RACE                        0
ETHNICITY                   0
GENDER                      0
BIRTHPLACE                  0
ADDRESS                     0
CITY                        0
STATE                       0
COUNTY                      0
ZIP                       545
LAT                         0
LON                         0
HEALTHCARE_EXPENSES         0
HEALTHCARE_COVERAGE         0
dtype: int64
```

In [8]:
```
print("\nMissing Values in Conditions Dataset:")
print(conditions.isnull().sum())
```

```
Missing Values in Conditions Dataset:
START             0
STOP           8169
PATIENT           0
ENCOUNTER         0
CODE              0
DESCRIPTION       0
dtype: int64
```

4. Check Duplicate Records

In [10]:
```
print("\nDuplicate Patient Records:",
patients.duplicated().sum())
```

```
Duplicate Patient Records: 0
```

In [11]:
```
print("Duplicate Condition Records:",
conditions.duplicated().sum())
```

```
Duplicate Condition Records: 0
```

5. Remove Duplicate Records

In [12]:
```
patients = patients.drop_duplicates()
conditions = conditions.drop_duplicates()
```

In [13]:
```
print("\nAfter Removing Duplicates:")
print("Patients:", patients.shape)
print("Conditions:", conditions.shape)
```

```
After Removing Duplicates:
Patients: (1163, 25)
Conditions: (38094, 6)
```

6. Convert Date Columns

In [14]:
```
patients["BIRTHDATE"] = pd.to_datetime( patients["BIRTHDATE"], errors="coerce"
```

In [15]:
```
patients["DEATHDATE"] = pd.to_datetime( patients["DEATHDATE"], errors="coerce"
conditions["START"] = pd.to_datetime( conditions["START"], errors="coerce" )
conditions["STOP"] = pd.to_datetime( conditions["STOP"], errors="coerce" )
```

7. Handle Missing Values

In [16]:
```
patients["ZIP"] = patients["ZIP"].fillna(0)
```

In [17]:
```
text_columns = [ "PREFIX", "SUFFIX", "MAIDEN", "MARITAL" ]
```

In [18]:
```
for column in text_columns: patients[column] = patients[column].fillna("Unknow
```

8. Calculate Patient Age

In [19]:
```
reference_date = pd.Timestamp.today()
```

In [20]:
```
patients["AGE"] = ( (reference_date - patients["BIRTHDATE"]) .dt.days / 365.25
```

9. Remove Invalid Ages

In [21]:
```
patients.loc[ (patients["AGE"] < 0) | (patients["AGE"] > 120), "AGE" ] = np.na
```

10. Count Conditions for Each Patient

In [22]:
```
condition_count = ( conditions .groupby("PATIENT") .size() .reset_index(name="
```

11. Find Unique Conditions

In [23]:
```
unique_conditions = ( conditions .groupby("PATIENT")["DESCRIPTION"] .nunique()
```

12. Integrate Patient and Condition Data

In [24]:
```
integrated_data = patients.merge( condition_count, left_on="Id", right_on="PAT
```

In [25]:
```
integrated_data = integrated_data.merge( unique_conditions, left_on="Id", righ
```

13. Handle Missing Condition Counts

In [26]:
```
integrated_data["CONDITION_COUNT"] = ( integrated_data["CONDITION_COUNT"] .fil
integrated_data["UNIQUE_CONDITIONS"] = ( integrated_data["UNIQUE_CONDITIONS"]
```

14. Create Age Groups

In [28]:
```
integrated_data["AGE_GROUP"] = pd.cut( integrated_data["AGE"],
                                       bins=[0, 18, 40, 60, 80, 120],
                                       labels=[ "Child", "Young Adult", "Middle
                                       include_lowest=True )
```

15. Transform Gender Values

In [29]:
```
integrated_data["GENDER_CODE"] = ( integrated_data["GENDER"] .map({ "M": 0, "F
```

16. Display Final Dataset

In [30]:
```
print("\nFinal Integrated and Transformed Dataset:")
display( integrated_data[ [ "Id", "FIRST", "LAST", "AGE", "AGE_GROUP", "GENDER
```

```
Final Integrated and Transformed Dataset:
```

	Id	FIRST	LAST	AGE	AGE_GR
0	b9c610cd-28a6-4636-ccb6-c7a0d2a4cb85	Damon455	Langosh790	7.0	
1	c1f1fcaa-82fd-d5b7-3544-c8f9708b06a8	Thi53	Wunsch504	21.0	Young A
2	339144f8-50e1-633e-a013-f361391c4cff	Chi716	Greenfelder433	28.0	Young A
3	d488232e-bf14-4bed-08c0-a82f34b6a197	Phillis443	Walter473	24.0	Young A
4	217f95a3-4e10-bd5d-fb67-0cfb5e8ba075	Jerrold404	Herzog843	33.0	Young A
5	faac724a-a9e9-be66-fe1e-3044dc0ba8ea	Brandon214	Watsica258	6.0	
6	23d16ee3-8cd4-eeb8-e77e-1e5fbf4c4159	Rodrigo242	Arellano2	36.0	Young A
7	aade3c61-92bd-d079-9d28-0b2b7fde0fbb	Sal878	Hoppe518	41.0	Middle
8	0288c42c-43a1-9878-4a9d-6b96caa12c40	Miki234	Kozey370	47.0	Middle
9	dc6c06d0-a7d8-100f-c08b-46c93700c188	Taylor21	Gulgowski816	20.0	Young A

17. Basic Statistics

In [31]:
```
print("\nBasic Statistics:")
display( integrated_data[ [ "AGE", "HEALTHCARE_EXPENSES", "HEALTHCARE_COVERAGE
```

```
Basic Statistics:
```

	AGE	HEALTHCARE_EXPENSES	HEALTHCARE_COVERAGE	CONDITION_
count	1163.000000	1.163000e+03	1.163000e+03	1163
mean	49.760103	1.256198e+06	1.718008e+05	32
std	27.590503	1.148120e+06	4.894650e+05	35
min	5.000000	8.259920e+02	0.000000e+00	0
25%	27.000000	4.426256e+05	1.293600e+03	12
50%	48.000000	1.142154e+06	1.747219e+04	27
75%	68.000000	1.900910e+06	2.051068e+05	43
max	115.000000	2.310000e+07	1.360000e+07	460

18. Display Most Common Conditions

In [34]:
```
print("\nTop 10 Most Common Medical Conditions:")
top_conditions = ( conditions["DESCRIPTION"] .value_counts() .head(10) )
print(top_conditions)
```

```
Top 10 Most Common Medical Conditions:
DESCRIPTION
Full-time employment (finding)                 13805
Stress (finding)                                5137
Part-time employment (finding)                  2426
Social isolation (finding)                      1243
Viral sinusitis (disorder)                      1233
Limited social contact (finding)                1200
Not in labor force (finding)                    1077
Victim of intimate partner abuse (finding)       819
Acute viral pharyngitis (disorder)               678
Acute bronchitis (disorder)                      571
Name: count, dtype: int64
```

19. Save Cleaned and Integrated Dataset

In [35]:
```
integrated_data.to_csv( "cleaned_integrated_healthcare_data.csv", index=False
print( "\nCleaned and integrated dataset saved successfully." )
```

```
Cleaned and integrated dataset saved successfully.
```